

Trabajo Práctico Integrador

UNQ-Shop

- Integrantes:

Erick Familia aquino
Alexis Lautaro Abrahan
Lautaro Santiago Huevo:

- Email:

familiaaquinoerick@gmail.com
alexislautaroabrahan1@gmail.com
lautarohuevo9@gmail.com

2.1 Catálogo de productos

Se eligió el **patrón Composite** porque el dominio requiere tratar de manera uniforme a los **productos individuales** y a los **paquetes**. Ambos representan elementos del catálogo y deben responder a las mismas operaciones (como obtener el nombre, la descripción y el precio), independientemente de si se trata de un producto simple o de un conjunto de elementos.

Además, la consigna establece que un paquete puede contener tanto productos como otros paquetes, formando una estructura jerárquica. El patrón Composite está diseñado precisamente para modelar relaciones de este tipo, permitiendo representar composiciones recursivas de objetos sin que se requiera distinguir entre un criterio simple y uno complejo.

2.2 Ciclo de vida del pedido

Se eligió el **patrón State** porque el comportamiento del pedido depende del estado en el que se encuentra durante su ciclo de vida. Cada estado habilita únicamente determinadas operaciones y transiciones, mientras que las acciones inválidas deben ser rechazadas mediante una excepción de dominio.

Además, la consigna define reglas de negocio específicas para cada estado, como la modificación del stock, la generación de reembolsos o la imposibilidad de realizar cambios una vez alcanzados los estados terminales. El patrón State permite encapsular estas reglas en objetos que representan cada estado, evitando concentrar toda la lógica en estructuras condicionales complejas.

2.3 Métodos de envío

Se eligió el **patrón Strategy** porque la consigna define distintas formas de calcular el costo y las condiciones de envío, donde cada método posee un algoritmo propio y requiere información diferente para realizar el cálculo.

Además, el pedido debe poder utilizar cualquiera de estos métodos de envío sin modificar su estructura ni conocer los detalles de cómo se calcula cada uno. El patrón Strategy permite encapsular cada algoritmo de cálculo en una estrategia independiente e intercambiable, manteniendo desacoplada la lógica del pedido de las distintas políticas de envío.

2.4 Métodos de pago

Se eligió el **patrón Template Method** porque la consigna define un proceso de pago que siempre sigue la misma secuencia de pasos, pero cuya implementación varía según el medio de pago utilizado.

Además, existen etapas obligatorias que cada medio debe implementar (validar datos, reservar fondos y ejecutar la transacción), mientras que otras poseen un comportamiento común que puede reutilizarse y personalizarse si es necesario, como la notificación del resultado. El patrón Template Method permite definir la estructura general del algoritmo en una clase base y delegar la implementación de los pasos específicos a las subclases.

2.5 Notificaciones del Pedido

Se eligió el **patrón Observer** porque la consigna requiere que múltiples subsistemas reaccionen automáticamente cuando un pedido cambia de estado, sin que el módulo de pedidos conozca ni dependa de ellos.

Además, cada subsistema responde de manera independiente y ante diferentes transiciones de estado, y se prevé la incorporación de nuevos observadores en el futuro. El patrón Observer permite que el pedido publique el evento de cambio de estado y que cada subsistema interesado decida cómo actuar, manteniendo un bajo acoplamiento entre el emisor del evento y sus receptores.

2.6 Búsqueda en el catálogo

Se eligió el **patrón Composite** porque la consigna establece que los criterios simples pueden combinarse mediante operadores lógicos (AND, OR y NOT), y que estas combinaciones también deben comportarse como criterios de búsqueda. Esto implica una estructura jerárquica en la que un criterio compuesto puede contener otros criterios simples o incluso otros criterios compuestos.

2.8 Reportes**

Se eligió el **patrón Visitor** porque la consigna requiere aplicar distintas operaciones de exportación sobre los mismos elementos del dominio sin modificar su estructura. La información del reporte debe mantenerse independiente del formato de salida, permitiendo representar los mismos datos en texto plano, CSV o HTML. El patrón Visitor permite

encapsular cada forma de procesamiento en un visitante distinto, separando la lógica de generación del reporte de la estructura de los objetos visitados.